

Chapter 06 .앙상블 분류 (랜덤 포레스트)

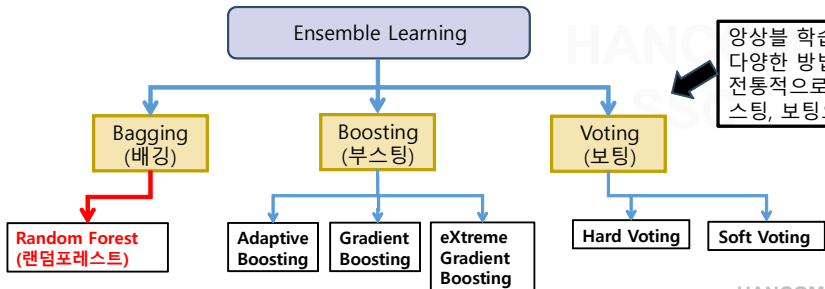
- Voting(보팅) 앙상블
- 배깅(Bagging Algorithm)
- 랜덤 포레스트 앙상블

앙상블 알고리즘(Ensemble Learning)

: 여러 개의 분류기(Classifier)를 생성하고 그 예측을 결합함으로써 보다 정확한 최종 예측을 도출하는 기법

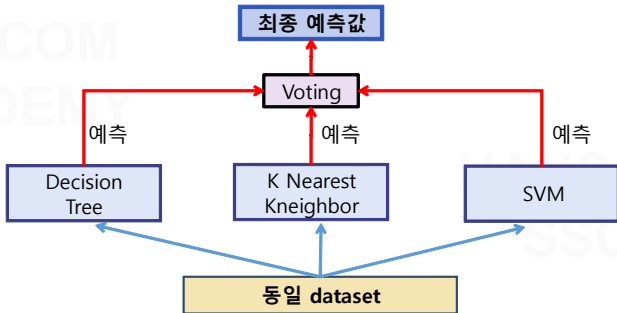
: 다양한 분류기의 예측 결과를 결합함으로써 단일 분류기의 신뢰성과 정확도 향상을 목표

- 이미지, 영상, 음성등 비정형 데이터 분류에는 딥러닝이 뛰어난 성능 발휘
- 대부분의 정형 데이터 분류 시에는 앙상블이 뛰어난 성능 발휘
- 앙상블 알고리즘의 대표 격인 **랜덤포레스트**, 그래디언트 부스팅 알고리즘 활용도가 높음



1. Voting(보팅) 앙상블

- : 여러 종류의 모델 알고리즘을 사용한 각각의 결과에 대해 투표를 통해 최종 결과를 예측하는 방식
- : 최종 예측 방법에 따라 하드보팅(Hard Voting), 소프트보팅(Soft Voting)으로 구분
- : 동일 dataset에 대해 서로 다른 모델별로 다른 예측결과를 가지고 투표 진행

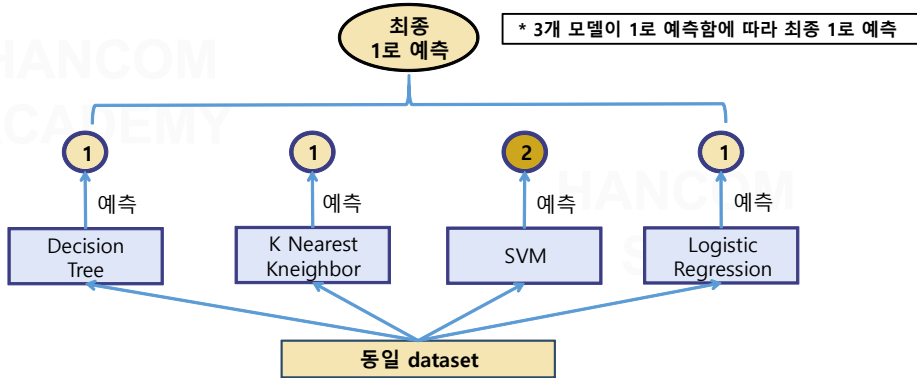


```
from sklearn.ensemble import VotingClassifier
```

1-1) Hard Voting

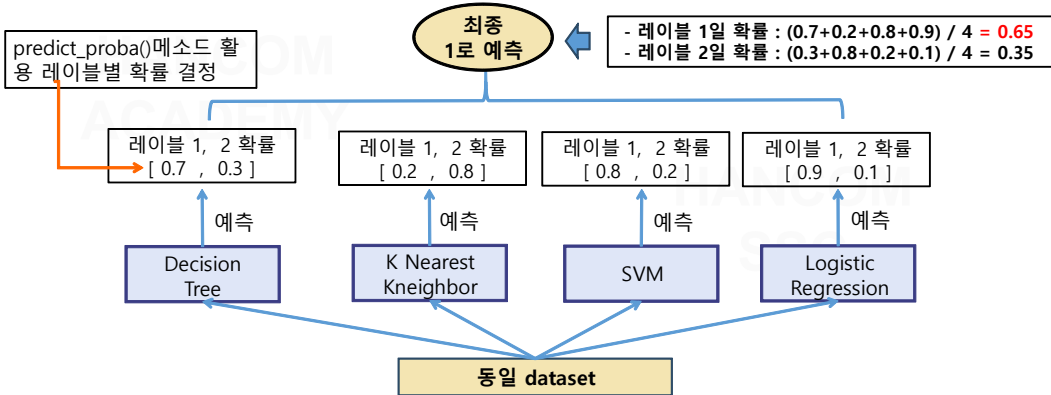
: 다수결 원칙과 비슷하게 다수결 투표 결과에 따라 최종 예측값 도출

: 여러 분류 모델 간 다수결로 최종 예측값(class) 결정



1-2) Soft Voting

- : 분류 모델기들의 레이블 값 결정 확률을 모두 더하고 이를 평균해서 이들 중 확률이 가장 높은 레이블 값을 최종 보팅 결과값으로 선정
- : 하드 보팅보다 좀 더 정교하며 성능이 높은 투표 방식, 일반적으로 소프트 보팅 사용



예제 1) 각 단일 모델 _ 손글씨분류 정확도 측정

```
from sklearn import datasets
from sklearn import tree
from sklearn.neighbors import KNeighborsClassifier
from sklearn.svm import SVC
```

```
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score
```

```
mnist = datasets.load_digits()
features = mnist['data'] # 8 * 8 = 총 64개 특징
labels = mnist['target']
print('labels : ', labels)
```

```
#import matplotlib.pyplot as plt
#plt.imshow(features[0].reshape(8,8))
#plt.show()
```

```
X_train, X_test, Y_train, Y_test = train_test_split(features, labels, test_size=0.2)
print(X_train[:1])
print(len(X_train))
print(len(X_test))
```

손 글씨로 쓴 숫자 데이터 셋
sklearn 8 * 8 image 데이터 제공

features[0] :

0	0	5	13	9	1	0	0	0	0	13	15	10	15	5	0	0	3
15	2	0	11	8	0	0	4	12	0	0	8	8	0	0	5	8	0
0	9	8	0	0	4	11	0	1	12	7	0	0	2	14	5	10	12
0	0	0	0	6	13	10	0	0	0								

labels : [0 1 2 ... 8 9 8]



훈련셋 분할

예제 1) 각 단일 모델 _ 손글씨분류 정확도 측정

각 단일 모델 정확도 측정

#tree 모델

max_features : 최적의 분할을 위해 고려할 최대 feature 개수

max_depth : 트리 성장 제한 기준, 몇단계 깊이 까지 성장시킬지 지정

```
model_tree = tree.DecisionTreeClassifier(criterion='gini', max_depth=8,  
                                         max_features=32,random_state=35)
```

```
model_tree.fit(X_train, Y_train)
```

```
tree_predicted = model_tree.predict(X_test)
```

```
print('tree accuracy : ', accuracy_score(Y_test,tree_predicted)) → tree accuracy : 0.8305555555555556
```

KNN 모델

```
model_knn = KNeighborsClassifier(n_neighbors=299)
```

```
model_knn.fit(X_train, Y_train)
```

```
knn_predicted = model_knn.predict(X_test)
```

```
print('knn accuracy : ', accuracy_score(Y_test, knn_predicted)) → knn accuracy : 0.8361111111111111
```

예제 1) 각 단일 모델 _ 손글씨분류 정확도 측정

각 단일 모델 정확도 측정

SVM 모델

probability = True : predict_proba() 함수 사용하기 위해 True 설정

model_SVC = SVC(C=0.1, gamma=0.003, probability=True, random_state=35)

model_SVC.fit(X_train, Y_train)

svm_predicted = model_SVC.predict(X_test)

print('svm accuracy : ', accuracy_score(Y_test, svm_predicted)) → svm accuracy : 0.8416666666666667

import numpy as np

np.set_printoptions(precision=6, suppress=True) # 지수표기법이 아닌 소수점 6자리까지 표현

분류값별 확률확인

svm_proba = model_SVC.predict_proba(X_test)

print(svm_proba[:2])

print(Y_test[:2]) → [0 2]

[[0.893896 0.009331 0.012385 0.008465 0.00829
8 0.017073 0.004852 0.01028
0.017244 0.018177]
[0.006574 0.030123 0.733911 0.014289 0.01709
0.025248 0.010297 0.021711
0.112042 0.028716]]

예제 2) 하드 보팅, 소프트 보팅 활용 정확도 측정

```
from sklearn import datasets
from sklearn import tree
from sklearn.neighbors import KNeighborsClassifier
from sklearn.svm import SVC
from sklearn.ensemble import VotingClassifier
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score

mnist = datasets.load_digits()
features = mnist['data'] # 8 * 8 = 총 64개 특징
labels = mnist['target']

X_train, X_test, Y_train, Y_test = train_test_split(features, labels, test_size=0.2)
print(X_train[:1])
print(len(X_train))
print(len(X_test))
```

각 단일 모델 정확도 측정

tree 모델

```
# max_features : 최적의 분할을 위해 고려할 최대 feature 개수
# max_depth : 트리 성장 제한 기준, 몇단계 깊이 까지 성장시킬지 지정
model_tree = tree.DecisionTreeClassifier(criterion='gini', max_depth=8,
                                         max_features=32, random_state=46)

model_tree.fit(X_train, Y_train)
tree_predicted = model_tree.predict(X_test)
print('tree accuracy : ', accuracy_score(Y_test, tree_predicted))
```

KNN 모델

```
model_knn = KNeighborsClassifier(n_neighbors=299)
model_knn.fit(X_train, Y_train)
knn_predicted = model_knn.predict(X_test)
print('knn accuracy : ', accuracy_score(Y_test, knn_predicted))
```

SVM 모델

```
# probability = True : predict_proba() 함수 사용하기 위해 True 설정
model_SVC = SVC(C=0.1, gamma=0.003, probability=True, random_state=46)
model_SVC.fit(X_train, Y_train)
svm_predicted = model_SVC.predict(X_test)
print('svm accuracy : ', accuracy_score(Y_test, svm_predicted))
```

하드보팅 정확도

weights : 발생회수에 대한 가중치

```
hardvoting_model = VotingClassifier(estimators=[  
    ('decision_tree', model_tree),  
    ('knn', model_knn),  
    ('svm', model_SVC)  
], weights=[1,1,1], voting='hard')  
hardvoting_model.fit(X_train, Y_train)
```

hardvoting accuracy : 0.9055555555555555
56

```
hardvoting_predicted = hardvoting_model.predict(X_test)  
print('hardvoting accuracy : ', accuracy_score(Y_test, hardvoting_predicted))
```

소프트보팅 정확도

weights : 발생회수에 대한 가중치

```
softvoting_model = VotingClassifier(estimators=[  
    ('decision_tree', model_tree),  
    ('knn', model_knn),  
    ('svm', model_SVC)  
], weights=[1,1,1], voting='soft')  
softvoting_model.fit(X_train, Y_train)
```

softdvoting accuracy : 0.9222222222222222
23

```
softdvoting_predicted = softvoting_model.predict(X_test)  
print('softdvoting accuracy : ', accuracy_score(Y_test, softdvoting_predicted))
```

단일 모델, 앙상블 모델 정확도 시각화

```
import matplotlib.pyplot as plt
```

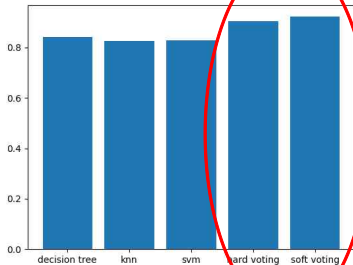
```
import numpy as np
```

```
x = np.arange(5)
```

```
plt.bar(x, height=[accuracy_score(Y_test, tree_predicted),  
                  accuracy_score(Y_test, knn_predicted),  
                  accuracy_score(Y_test, svm_predicted),  
                  accuracy_score(Y_test, hardvoting_predicted),  
                  accuracy_score(Y_test, softdvoting_predicted)])
```

```
plt.xticks(x, ['decision tree', 'knn', 'svm', 'hard voting', 'soft voting'])
```

```
plt.show()
```



2. 배깅(Bagging Algorithm) : Bootstrap + Aggregating

- 한 가지 분류 모델을 여러 개 만들어서 서로 다른 훈련데이터로 학습시킨 후(bootstrap) 서로 다른 예측값들을 집계하여(Aggregating) 가장 높은 예측값으로 결과 도출하는 앙상블
- aggregating(결과 결합) : 각 샘플의 결과변수(예측값)을 결합하는 방법으로 연속형(회귀)이면 평균(Average), 범주형(분류)이면 다중 투표(majority vote) 활용

* 부트스트랩(Bootstrap)

- 원래 데이터 셋으로부터 관측치를 반복적으로 랜덤 추출(복원 반복 랜덤 추출)하여 데이터 셋을 얻는 방법
- 데이터 양을 임의적으로 늘리고, 데이터 셋의 분포가 고르지 않을 때 고르게 만드는 효과
- N개의 데이터를 총 K개의 데이터로 나누고 나눌때 데이터의 중복을 허용

* 의사결정 트리 6개 배깅할 경우의 부트스트랩 결과

Original Data : [1,2,3,4,5,6,7,8,9]



의사결정 트리1의 학습 데이터 : [1,2,3,4,5,1,2,4,9]
의사결정 트리2의 학습 데이터 : [2,3,4,5,6,2,8,7,3]
의사결정 트리3의 학습 데이터 : [3,4,5,7,8,3,9,7,2]
의사결정 트리4의 학습 데이터 : [5,6,3,4,5,7,1,2,9]
의사결정 트리5의 학습 데이터 : [8,2,9,4,5,3,6,5,7]
의사결정 트리6의 학습 데이터 : [6,7,8,4,5,2,1,3,8]

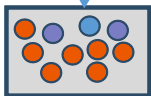
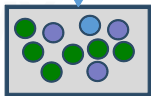
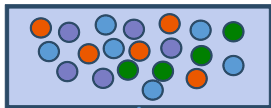
배깅 : 하나의 모델을 다양하게 학습시킴!!

* 대표적인 Bagging 알고리즘 : 랜덤 포레스트(Random Forest) <= 여러개 나무가 모여 있다!!(Forest)

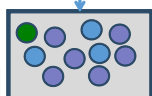
- 각각의 의사결정트리가 독립적 병렬 학습화

2-1) 배깅(Bagging Algorithm) 도식화

하나의 모델로
각각의 데이터 학습



...



★
(부트스트랩)

Decision
Tree 동일
모델 적용

Decision
Tree Model1

Decision
Tree Model2

Decision
Tree Model3

...

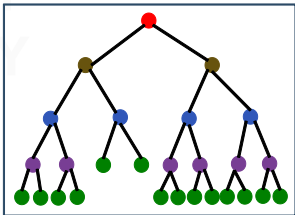
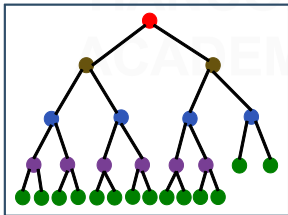
Decision
Tree ModelN

Aggregating
(Majority Vote, Average)

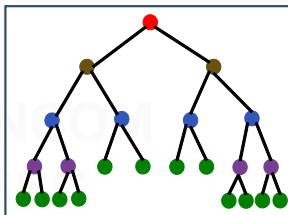
Output

3. 랜덤 포레스트 앙상블

- 랜덤하게 선택된 샘플과 특성을 사용하여 여러개의 의사결정 트리 생성 및 훈련
- **부트스트랩 샘플 생성**(n 개 샘플중 랜덤하게 n 횟수만큼 반복 추출(데이터 중복 허용))
- 모든 특징을 사용해 트리 분기를 결정하는 의사결정트리와 달리
각 노드 분할 테스트에서 특징 변수를 무작위로 선택함으로써 트리의 다양성 확보 및 **과대적합 해결**



....



수 많은 의사결정트리가 모여 만들어진 숲!!

3-1) 랜덤 포레스트 특징

- 의사결정트리의 과적합 현상을 개선하기 위해 고안된 앙상블 기법
- 각각의 트리가 독립적으로 학습하기 때문에 학습과정을 병렬화할 수 있음
- 타깃 예측을 잘하며 각각이 구별되는 여러개의 트리를 만들기 위해 무작위성이 부여됨
- **별다른 파라미터 조정 없이도 일반화 및 우수한 성능을 발휘하며 특징변수의 정규화가 필요 없다!**
- 대표적인 '배깅' 모델
- 트리의 개수가 많아질 수록 시간이 더 오래 걸림
- 어떤 변수들이 어떻게 영향을 미치는지 설명이 가능한 비즈니스 문제 해결에 유용
(model.feature_importances_ : 변수 중요도 출력)

3-2) 랜덤 포레스트 과정

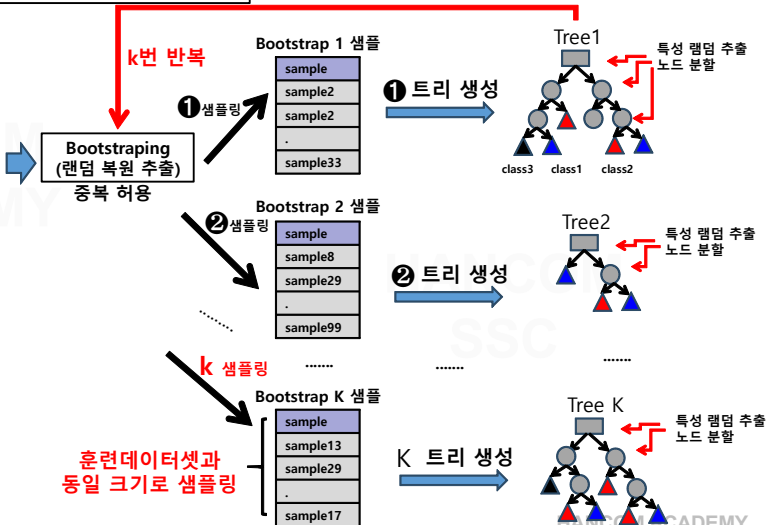
- 1) 모델 내부에 트리를 만들기 위해 주어진 트레이닝 데이터 세트에서 무작위로 중복을 허용해서 전체 훈련 샘플중 **n개 샘플** 선택한 부트스트랩 샘플 생성
(**n 값** : scikit-learn API는 원래 훈련데이터 전체 개수와 동일한 수를 할당)
- 2) 선택한 **n개의 부트스트랩 샘플**에서 데이터 특성값(꽃잎너비, 꽃잎길이등의 특성)을 중복 허용없이 **d개 선택** (**d개수는 default : $\sqrt{m}$**), **m** : 훈련데이터의 전체 특성 수
- 3) 이를 기반으로 훈련과정에서 의사결정트리를 학습하고 생성 (**정보이득 기준으로 분할 기준 학습**)

- 4) 1 ~ 3 번 단계를 k번 반복 (k 값은 **n_estimators** 파라미터 설정값 , default 100)
- 5) 1~ 4 단계를 통해 **생성된 모든 트리를 기반으로 예측 수행**, 회귀의 경우 모든 예측의 평균하여 결과 도출, 분류의 경우 soft voting 응용 방식으로 사용

3-3) 랜덤 포레스트 훈련 과정 도식화

sample	특성1		특성j	class
sample1	0.5	.	0.4	class1
sample2	0.8	.	0.7	class1
.	.	.	.	class1
.	.	.	.	class2
.	.	.	.	class2
.	.	.	.	class2
.	.	.	.	class3
sample99	0.7	.	0.5	class3
sample100	0.6	.	0.8	class3

원본 훈련데이터셋

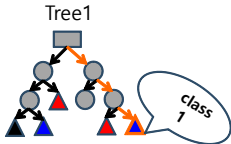


3-4) 랜덤 포레스트 예측 과정 도식화

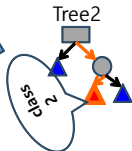
예측 테스트 샘플

test sample
sample33
sample2
.
sample17

생성된 1번
트리에 주입



생성된 2번
트리에 주입

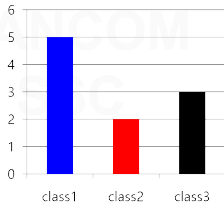


.....

생성된 k번
트리에 주입



voting 결과 도출



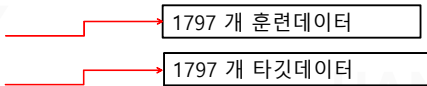
3-5) 랜덤 포레스트 주요 파라미터

- **n_estimators** : 모델에서 사용할 트리 개수 (학습시 생성할 트리 개수, default : 100)
- **criterion** : 분할 품질을 측정하는 기능 (default : gini)
- **max_depth** : 트리의 최대 깊이
- **min_samples_split** : 내부 노드를 분할하는데 필요한 최소 샘플 수 (default : 2)
- **min_samples_leaf** : 리프 노드에 있어야할 최소 샘플 수 (default : 1)
- **max_features** : 각 노드에서 분할에 사용할 특징의 최대 수 (default : auto) ★
 - If "auto", then ``max_features=sqrt(n_features)` : 제공근'
 - 전체 특성의수로 설정하면 모든 특성을 고려하므로 decision tree 무작위성이 들어가지 않음, 하지만 부트스트랩의 무작위 샘플링은 존재
 - **보통 Default 사용**
- **max_leaf_nodes** : 리프 노드의 최대수
- **min_impurity_decrease** : 최소 불순도
- **min_impurity_split** : 트리 성장을 멈출 임계치
- **bootstrap** : 부트스트랩(중복허용 샘플링) 사용 여부 (default=True)
- **oob_score** : 부트스트랩 샘플에 포함되지 않고 남는 샘플(OOB:out of bag)을 사용하여 트리 평가
- **n_jobs** : 병렬 실행에 사용할 CPU 코어 수 지정 , **-1로 설정해 시스템에 있는 모든 코어 사용**
- **random_state** : 동일한 결과를 얻기 위한 난수 seed 설정
- **verbose** : 실행 과정 출력 여부

예제 1) 랜덤 포레스트_의사결정트리 정확도 성능 비교

```
from sklearn.datasets import load_digits
import matplotlib.pyplot as plt
from sklearn import tree
from sklearn.ensemble import RandomForestClassifier
mnist = load_digits() # 0~9 손글씨 숫자 데이터 셋 ( 분류용 )
```

```
print(mnist['data'][:3])
print(len(mnist['data']))
print(mnist['target'])
print(len(mnist['target']))
```



1797 개 훈련데이터

1797 개 타깃데이터

```
features = mnist['data'] # 1797 개의 8 * 8 이미지데이터셋
labels = mnist['target']
```

예제 1) 랜덤 포레스트_의사결정트리 정확도 성능 비교

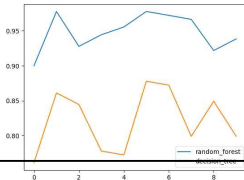
```
from sklearn.model_selection import cross_validate #교차검증
RFmodel = RandomForestClassifier()
RF_scores = cross_validate(RFmodel, features, labels, cv=10) # 랜덤포레스트모델 10-fold 교차 검증
print(RF_scores['test_score']) # 랜덤포레스트 앙상블 검증평가 점수

DT_scores = cross_validate(tree.DecisionTreeClassifier(), features, labels, cv=10) #의사결정트리 10-fold 교차검증
print(DT_scores['test_score']) # 의사결정트리 검증평가 점수

import numpy as np
print('random_forest accuracy : ',np.mean(RF_scores['test_score']))
print('decision tree accuracy : ',np.mean(DT_scores['test_score']))
# 랜덤 포레스트 앙상블이 별도의 하이퍼파라미터 설정 없는 의사결정트리 보다 월등히 높은 성능 발휘

import pandas as pd
df = pd.DataFrame({'random_forest':RF_scores['test_score'],
                  'decision_tree':DT_scores['test_score']})

print(df)
df.plot()
plt.show()
```



예제 2) 랜덤 포레스트_손글씨 분류 예측

```
from sklearn.datasets import load_digits
import matplotlib.pyplot as plt
from sklearn import tree
from sklearn.ensemble import RandomForestClassifier
mnist = load_digits() # 0~9 손글씨 숫자 데이터 셋 (분류용)
print(mnist['data'][-3:])
print(mnist['target'][-3:])
```

```
features = mnist['data'] # 1797 개의 8 * 8 이미지데이터셋
labels = mnist['target']
```

```
RFmodel = RandomForestClassifier()
RFmodel.fit(features, labels)
```

학습시 라벨(labels)은 분할 기준(규칙)을
평가(결정)할 기준이 됨, 즉 정보이득(불순도 감소)량을 계산해 가장 좋은
분할 기준을 선택(결정)하는 역할

```
print('score : ', RFmodel.score(features, labels))
```

score : 1.0

```
predicted = RFmodel.predict(features[-5:])
print('labels : ', labels[-5:])
print('pre : ', predicted)
```

labels : [9 0 8 9 8]

pre : [9 0 8 9 8]

예제 2) 랜덤 포레스트_손글씨 임의 데이터 예측

```
tempdata = [ 0. , 0. ,10. ,14. , 8. , 1. , 0. ,0. ,0. ,2. ,16. ,14. ,6. ,15. , 6. ,0. , 0. , 0.,  
12. ,15. , 8. ,15. , 0. , 0. , 0. , 15. ,5. ,16. ,16. ,10. ,0. ,0. ,0. ,0. ,12. ,15.,  
13. ,12. , 0. , 0. , 0. , 4. ,16. , 5. ,4. ,16. ,6. ,0. ,0. ,8. ,16. ,10. ,8. ,16.,  
8. , 1. , 0. , 1. , 7. ,12. ,14. ,12. ,1. ,0.]
```

```
import numpy as np  
import matplotlib.pyplot as plt  
temparr = np.array(tempdata) # reshape 하기위해 numpy array로 변환  
print(temparr)  
temp_pred = RFmodel.predict(temparr)
```

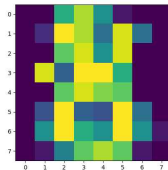
```
print('temp_pred : ', temp_pred)
```

임의 데이터 8
로 예측

temp_pred : [8]

```
plt.imshow(temparr.reshape(8,8)) # 디폴트 색상 viridis  
plt.show()
```

실제 데이터 시각화



Thank You